

Kernel Density Estimation

Codice annotato per la presentazione — tre slide

23 aprile 2026

Indice

1	Introduzione teorica	2
1.1	Cos'è la KDE	2
1.2	Il ruolo della bandwidth h	2
1.3	Distribuzione generatrice usata nel notebook	2
1.4	Metodi automatici per la scelta di h	3
2	Setup comune	4
3	Slide 1 — Bias, Varianza e curva MISE	5
3.1	Obiettivo	5
3.2	Come si calcola il MISE empiricamente	5
3.3	Grafico	6
4	Slide 2 — DGP e variazione di h	8
4.1	Obiettivo	8
4.2	Calcolo dell'ISE e dell' h ottimale	8
4.3	Pannello 1 — DGP	8
4.4	Funzione helper per i pannelli KDE	9
4.5	Pannelli 2-4 — Tre valori di h	10
4.6	Layout combinato (4 pannelli — pronto per la slide)	12
5	Slide 3 — Confronto selettori di h e curva ISE	13
5.1	Obiettivo	13
5.2	Calcolo dei selettori	13
5.3	Tabella formattata	13
5.4	Curva MISE con linee verticali dei selettori	14
6	Riepilogo	17

1 Introduzione teorica

1.1 Cos'è la KDE

La **Kernel Density Estimation (KDE)** è un metodo non parametrico per stimare la densità di probabilità $f(x)$ di una variabile aleatoria continua, a partire da un campione osservato $x_1, \dots, x_n$.

A differenza dell'istogramma, non dipende dalla scelta dei bin e produce una curva continua e differenziabile.

Definizione. Data una funzione kernel $K(\cdot)$ e una bandwidth $h > 0$:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

Il kernel K deve soddisfare: $K(u) \geq 0$, $K(u) = K(-u)$ (simmetria), $\int K(u) du = 1$ (normalizzazione).

In questo notebook usiamo il **kernel gaussiano**:

$$K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$$

Ogni osservazione x_i contribuisce alla stima con una “campana” gaussiana centrata in x_i e di ampiezza h ; la stima finale è la loro media.

1.2 Il ruolo della bandwidth h

Il parametro h determina la *larghezza* di ogni campana e quindi il grado di lisciamiento:

- h **piccolo** $\Rightarrow$ ogni osservazione forma un picco distinto; la stima insegue il rumore campionario (**overfitting** / undersmoothing).
- h **grande** $\Rightarrow$ i picchi si fondono; la stima non coglie la struttura vera (**underfitting** / oversmoothing).

Il tradeoff si formalizza scomponendo il **MISE** (Mean Integrated Squared Error):

$$\text{MISE}(\hat{f}) = \mathbb{E} \left[\int (\hat{f}(x) - f(x))^2 dx \right] = \underbrace{\int \text{Bias}^2[\hat{f}(x)] dx}_{\uparrow \text{ con } h} + \underbrace{\int \text{Var}[\hat{f}(x)] dx}_{\downarrow \text{ con } h}$$

Poiché nella pratica f è sconosciuta, il MISE non è calcolabile direttamente. In questo notebook, essendo in contesto simulato, lo stimiamo empiricamente tramite **media Monte Carlo** su B campioni indipendenti — si veda la Sezione 3.

Il valore ottimale teorico scala come $h^* \propto n^{-1/5}$, che bilancia i due termini.

1.3 Distribuzione generatrice usata nel notebook

Usiamo una **mistura bimodale**, volutamente difficile per la KDE a bandwidth fissa:

$$f(x) = 0.4 \cdot \mathcal{N}(-2, 1^2) + 0.6 \cdot \mathcal{N}(3, 0.5^2)$$

I due picchi hanno varianze molto diverse (1 vs 0.25): una bandwidth unica non può adattarsi perfettamente a entrambi, rendendo il caso ideale per illustrare i limiti della KDE standard.

1.4 Metodi automatici per la scelta di h

Nella pratica la vera f è sconosciuta, quindi h^* non è calcolabile direttamente. Esistono diversi stimatori automatici:

Metodo	Idea	Funzione R
h^* (MISE empirico)	Minimo della curva MISE(h) stimata su B campioni Monte Carlo — referimento ottimale, disponibile solo in contesto simulato	—
Silverman	$1.06 \hat{\sigma} n^{-1/5}$ — ottimale se i dati sono gaussiani	<code>bw.nrd0()</code>
Scott	$1.059 \hat{\sigma} n^{-1/5}$ — variante di Silverman	<code>bw.nrd()</code>
Sheather–Jones	Plug-in adattivo; robusto su distribuzioni non gaussiane	<code>bw.SJ()</code>
UCV	Unbiased Cross-Validation; minimizza una stima non distorta del MISE	<code>bw.ucv()</code>
BCV	Biased Cross-Validation; variante con minore variabilità	<code>bw.bcv()</code>

Poiché in questo notebook lavoriamo in contesto simulato e conosciamo la vera f , possiamo calcolare h^* esattamente (come argmin della curva $\widehat{\text{MISE}}(h)$) e usarlo come **benchmark** per valutare quanto ogni selettore automatico si avvicini all’ottimo. Silverman e Scott tendono a *sovrastimare* h su distribuzioni multimodali, perché $\hat{\sigma}$ è gonfiata dalla distanza tra i picchi.

2 Setup comune

Questo blocco definisce le funzioni e i parametri condivisi da tutti i grafici successivi. Deve essere eseguito per primo.

```
set.seed(42)

# Vera densità (ground truth)
true_density <- function(x) {
  0.4 * dnorm(x, mean = -2, sd = 1) +
  0.6 * dnorm(x, mean = 3, sd = 0.5)
}

# Campionamento dalla mistura
sample_mixture <- function(n) {
  comp <- sample(c(1L, 2L), size = n, replace = TRUE, prob = c(0.4, 0.6))
  ifelse(comp == 1L, rnorm(n, -2, 1), rnorm(n, 3, 0.5))
}

# KDE con kernel gaussiano - implementazione manuale
# x_grid : vettore di punti in cui valutare la stima
# data   : campione osservato
# h      : bandwidth
kde_manual <- function(x_grid, data, h) {
  sapply(x_grid, function(x) mean(dnorm((x - data) / h)) / h)
}

# Parametri globali
n <- 500
data <- sample_mixture(n) # campione fisso per tutta l'analisi
x_grid <- seq(-6, 6, length.out = 1000) # griglia più compatta (velocità)
dx <- diff(x_grid)[1] # passo della griglia (usato per integrare)

df_true <- data.frame(x = x_grid, y = true_density(x_grid))
```

3 Slide 1 — Bias, Varianza e curva MISE

3.1 Obiettivo

La prima slide è **teorica**: mostra visivamente perché esiste un h ottimale, disegnando le tre curve — $\text{Bias}^2(h)$, $\text{Varianza}(h)$ e $\text{MISE}(h)$ — come funzione di h , stimate empiricamente tramite simulazione Monte Carlo.

3.2 Come si calcola il MISE empiricamente

Il MISE è il **valore atteso dell'ISE** rispetto a tutti i possibili campioni di taglia n :

$$\text{MISE}(h) = \mathbb{E}[\text{ISE}(h)] = \mathbb{E}\left[\int (\hat{f}(x) - f(x))^2 dx\right]$$

Poiché conosciamo la vera f (siamo in contesto simulato), possiamo stimarlo con la media campionaria su B simulazioni indipendenti:

$$\widehat{\text{MISE}}(h) = \frac{1}{B} \sum_{b=1}^B \text{ISE}_b(h), \quad \text{ISE}_b(h) = \int [\hat{f}_b(x) - f(x)]^2 dx$$

Allo stesso modo stimiamo il Bias^2 e la Varianza puntuali, poi integriamo:

$$\widehat{\text{Bias}}^2(h) = \int [\bar{\hat{f}}(x) - f(x)]^2 dx, \quad \bar{\hat{f}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

$$\widehat{\text{Var}}(h) = \int \frac{1}{B} \sum_{b=1}^B [\hat{f}_b(x) - \bar{\hat{f}}(x)]^2 dx$$

e si verifica che $\widehat{\text{MISE}} \approx \widehat{\text{Bias}}^2 + \widehat{\text{Var}}$.

```
# Parametri della simulazione
# Aumentare B (es. 500) e length.out (es. 80) per curve più lisce,
# a costo di maggior tempo di calcolo.
B <- 100 # numero di campioni Monte Carlo
h_seq <- seq(0.01, 2.5, length.out = 200) # griglia di h (piu punti per curva MISE piu liscia)

# Pre-calcolo della densità vera sulla griglia (costante, inutile ricalcolarla
# ad ogni iterazione del doppio ciclo)
p_true_vec <- true_density(x_grid)

# Simulazione: ciclo UNICO che accumula ISE e fhat_sum insieme
# Ottimizzazioni rispetto alla versione originale:
# 1. density() usa FFT in C - ordini di grandezza più veloce di kde_manual()
# che eseguiva un sapply() in R puro su tutti i 1000 punti della griglia.
# 2. Un solo ciclo Monte Carlo invece di due: ISE e fhat_sum vengono
# calcolati nella stessa iterazione b, eliminando il secondo set.seed(123)
# e il secondo loop da B x 100 chiamate.
# 3. p_true_vec pre-calcolata fuori dal loop (1000 x B x 100 chiamate in meno
# a true_density).
set.seed(123)
ise_mat <- matrix(NA_real_, nrow = B, ncol = length(h_seq))
fhat_sum <- matrix(0, nrow = length(x_grid), ncol = length(h_seq))
```

```

for (b in seq_len(B)) {
  samp <- sample_mixture(n)
  for (j in seq_along(h_seq)) {
    # density() valuta la KDE sulla stessa griglia x_grid (from/to/n allineati)
    fhat <- density(samp, bw = h_seq[j],
                    from = x_grid[1], to = x_grid[length(x_grid)],
                    n = length(x_grid))$y
    ise_mat[b, j] <- sum((fhat - p_true_vec)^2) * dx # integrazione numerica dell'ISE
    fhat_sum[, j] <- fhat_sum[, j] + fhat           # accumulo per la media
  }
  if (b %% 200 == 0) cat(sprintf(" Completate %d / %d simulazioni\n", b, B))
}

# MISE empirico = media degli ISE sui B campioni
mise_emp <- colMeans(ise_mat)

# Bias2 e Varianza empirici
fhat_mean <- fhat_sum / B # f̂(x) per ogni h - stimatore del valore atteso di f

bias2_emp <- numeric(length(h_seq))
for (j in seq_along(h_seq)) {
  bias2_emp[j] <- sum((fhat_mean[, j] - p_true_vec)^2) * dx
}

# Varianza = MISE - Bias2 (decomposizione esatta)
var_emp <- mise_emp - bias2_emp

h_mise_opt <- h_seq[which.min(mise_emp)]
cat(sprintf("h ottimale MISE empirico (B = %d): %.4f\n", B, h_mise_opt))

```

```
## h ottimale MISE empirico (B = 100): 0.1977
```

3.3 Grafico

```

df_curve <- rbind(
  data.frame(h = h_seq, y = bias2_emp, Componente = "Bias\U00B2"),
  data.frame(h = h_seq, y = var_emp, Componente = "Varianza"),
  data.frame(h = h_seq, y = mise_emp, Componente = "MISE")
)
df_curve$Componente <- factor(df_curve$Componente,
                             levels = c("Bias\U00B2", "Varianza", "MISE"))

col_curve <- c("Bias\U00B2" = "#e74c3c",
              "Varianza" = "#3498db",
              "MISE" = "#2c3e50")

lwd_curve <- c("Bias\U00B2" = 0.9, "Varianza" = 0.9, "MISE" = 0.75)
lty_curve <- c("Bias\U00B2" = "solid", "Varianza" = "solid", "MISE" = "dashed")

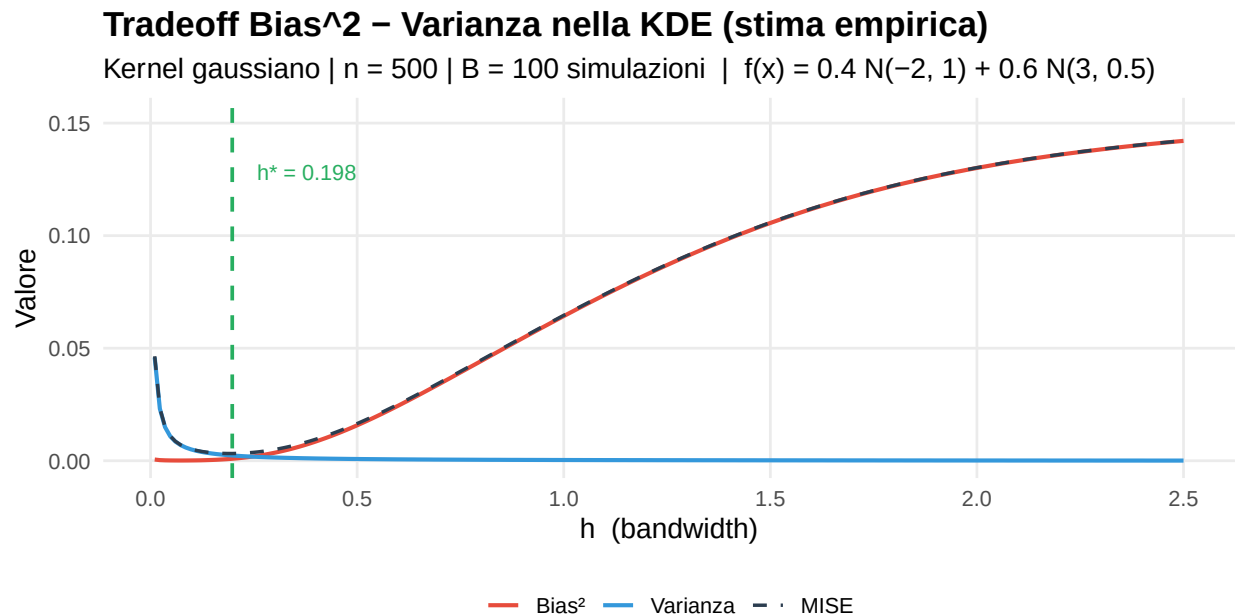
ggplot(df_curve, aes(x = h, y = y,
                     colour = Componente,
                     linewidth = Componente,
                     linetype = Componente)) +

```

```

geom_line() +
geom_vline(xintercept = h_mise_opt,
           colour = "#27ae60", linetype = "dashed", linewidth = 0.8) +
annotate("text",
         x = h_mise_opt + 0.06, y = max(mise_emp) * 0.90,
         label = paste0("h* = ", round(h_mise_opt, 3)),
         colour = "#27ae60", size = 3.5, hjust = 0) +
scale_colour_manual(values = col_curve) +
scale_linewidth_manual(values = lwd_curve) +
scale_linetype_manual(values = lty_curve) +
scale_y_continuous(limits = c(0, max(mise_emp) * 1.08)) +
labs(
  title = "Tradeoff Bias^2 - Varianza nella KDE (stima empirica)",
  subtitle = paste0("Kernel gaussiano | n = ", n, " | B = ", B,
                    " simulazioni | f(x) = 0.4 N(-2, 1) + 0.6 N(3, 0.5)",
  x = "h (bandwidth)",
  y = "Valore",
  colour = NULL,
  linewidth = NULL,
  linetype = NULL
) +
theme_minimal(base_size = 13) +
theme(
  plot.title = element_text(face = "bold"),
  legend.position = "bottom",
  panel.grid.minor = element_blank()
)

```



Come leggere il grafico. La curva rossa (Bias²) cresce con h : una finestra larga appiattisce i picchi. La curva blu (Varianza) decresce con h : più osservazioni contribuiscono a ogni stima. La curva nera (MISE) è la loro somma e ha la caratteristica **forma a U**: il suo minimo (linea verde) è l' h ottimale per questo n e questa distribuzione, stimato su B simulazioni Monte Carlo.

4 Slide 2 — DGP e variazione di h

4.1 Obiettivo

La seconda slide ha **quattro pannelli**:

1. Il Data Generating Process (vera densità + campione)
2. KDE con h troppo piccolo — overfitting
3. KDE con h ottimale (minimo ISE sul campione)
4. KDE con h troppo grande — underfitting

In ogni pannello KDE la ground truth è mostrata in tratteggio per permettere il confronto diretto.

4.2 Calcolo dell'ISE e dell' h ottimale

In contesto simulato possiamo calcolare l'**ISE** (Integrated Squared Error) direttamente, perché conosciamo f :

$$\text{ISE}(h) = \int [\hat{f}(x) - f(x)]^2 dx \approx \sum_j [\hat{f}(x_j) - f(x_j)]^2 \Delta x$$

```
h_grid <- seq(0.05, 2.5, length.out = 300)
ise_grid <- sapply(h_grid, function(h) {
  fhat <- kde_manual(x_grid, data, h)
  sum((fhat - true_density(x_grid))^2) * dx
})
h_opt <- h_grid[which.min(ise_grid)]
ise_opt <- min(ise_grid)

cat(sprintf("h ottimale (min ISE sul campione): %.4f\n", h_opt))

## h ottimale (min ISE sul campione): 0.2057

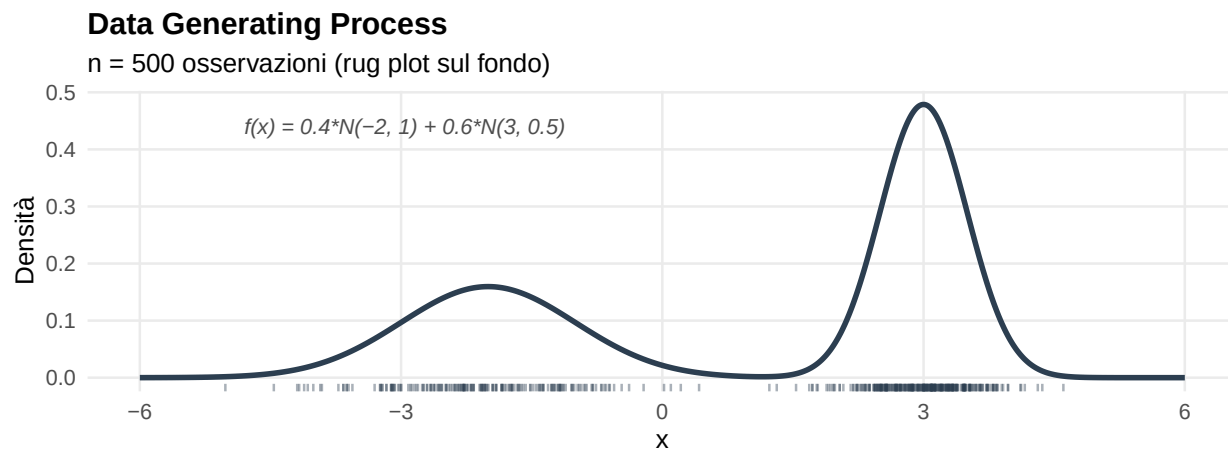
cat(sprintf("ISE minimo: %.6f\n", ise_opt))

## ISE minimo: 0.001649
```

4.3 Pannello 1 — DGP

```
p_dgp <- ggplot() +
  geom_rug(aes(x = data),
    alpha = 0.4, colour = "#2c3e50",
    length = unit(0.025, "npc")) +
  geom_line(data = df_true, aes(x = x, y = y),
    colour = "#2c3e50", linewidth = 1.2) +
  annotate("text", x = -4.8, y = 0.44,
    label = "f(x) = 0.4*N(-2, 1) + 0.6*N(3, 0.5)",
    size = 3.5, hjust = 0, colour = "gray30", fontface = "italic") +
  labs(
    title = "Data Generating Process",
    subtitle = paste0("n = ", n, " osservazioni (rug plot sul fondo)",
    x = "x", y = "Densit\u00e0")
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"),
    panel.grid.minor = element_blank())
```

p_dgp



4.4 Funzione helper per i pannelli KDE

```
# Costruisce un singolo pannello KDE vs ground truth
# h          : bandwidth da usare
# titolo     : titolo del pannello (stringa)
# sottotit   : sottotitolo (stringa)
# colore     : colore della curva KDE
panel_kde <- function(h, titolo, sottotit, colore) {
  df_kde <- data.frame(
    x = x_grid,
    y = density(
      data,
      bw = h,
      from = x_grid[1],
      to = x_grid[length(x_grid)],
      n = length(x_grid)
    )$y
  )

  ggplot() +
    # Area sotto la curva KDE
    geom_ribbon(data = df_kde,
              aes(x = x, ymin = 0, ymax = y),
              fill = colore, alpha = 0.12) +
    # Stima KDE
    geom_line(data = df_kde,
              aes(x = x, y = y, linetype = "KDE"),
              colour = colore, linewidth = 1.05) +
    # Ground truth (tratteggio nero) sopra la KDE per massima visibilità
    geom_line(data = df_true,
              aes(x = x, y = y, linetype = "Ground truth"),
              colour = "black", linewidth = 0.9) +
    scale_linetype_manual(
      values = c("Ground truth" = "dashed", "KDE" = "solid"),
      name = NULL
    )
}
```

```

) +
coord_cartesian(ylim = c(0, 0.67)) +
labs(title = titolo,
      subtitle = sottotit,
      x = "x", y = "Densit\u00e0") +
theme_minimal(base_size = 11) +
theme(
  plot.title = element_text(face = "bold", colour = colore, size = 11),
  plot.subtitle = element_text(size = 8.5, colour = "gray40"),
  legend.position = "bottom",
  legend.key.width = unit(1.2, "cm"),
  panel.grid.minor = element_blank()
)
}

```

4.5 Pannelli 2-4 — Tre valori di h

```

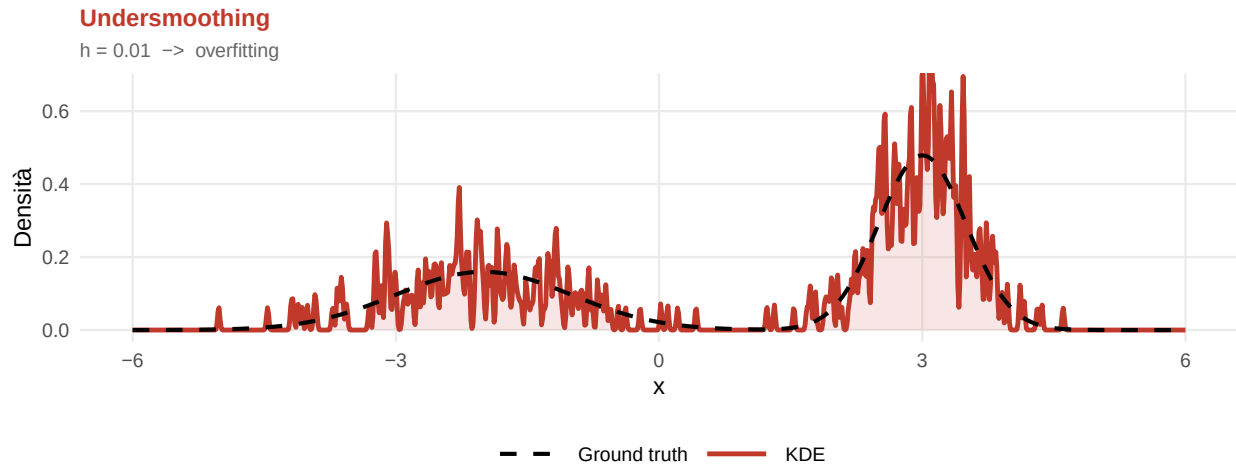
p_small <- panel_kde(
  h = 0.01,
  titolo = "Undersmoothing",
  sottotit = "h = 0.01 -> overfitting",
  colore = "#c0392b"
)

p_opt_panel <- panel_kde(
  h = 0.30,
  titolo = "Buona scelta",
  sottotit = "h = 0.30 -> buon fit",
  colore = "#27ae60"
)

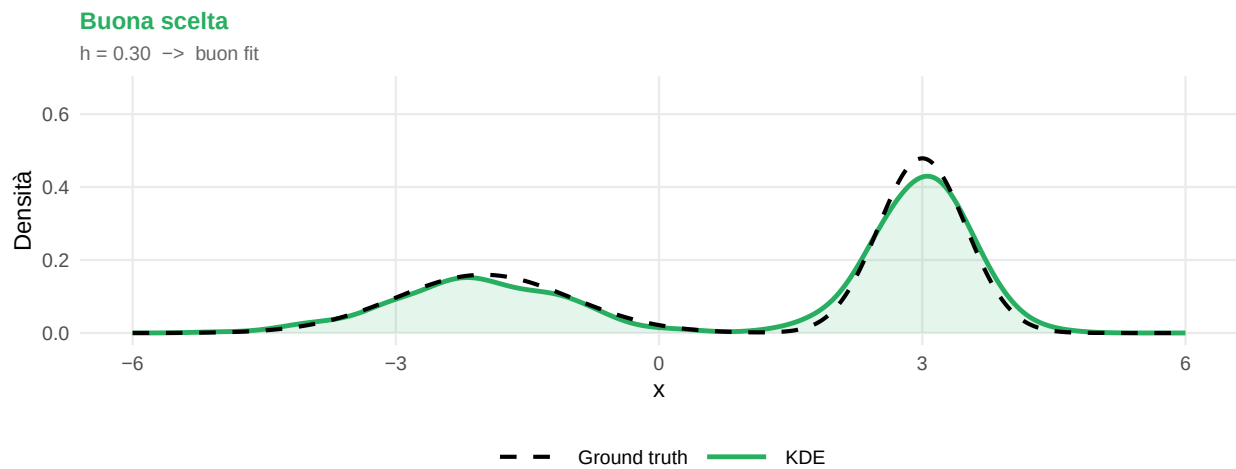
p_large <- panel_kde(
  h = 2.00,
  titolo = "Oversmoothing",
  sottotit = "h = 2.00 -> underfitting",
  colore = "#8e44ad"
)

# Stampa singoli pannelli per verifica
p_small

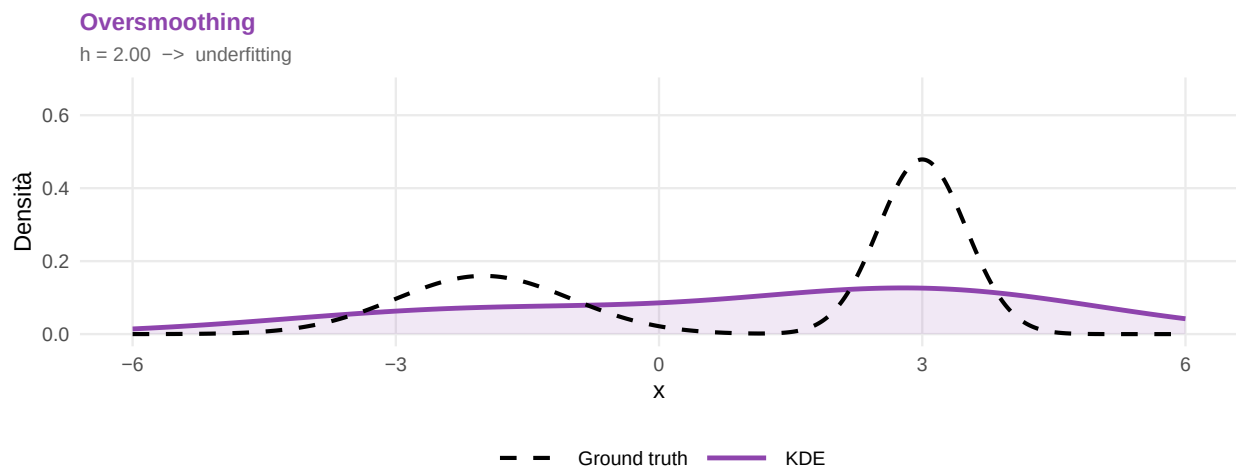
```



p_opt_panel



p_large



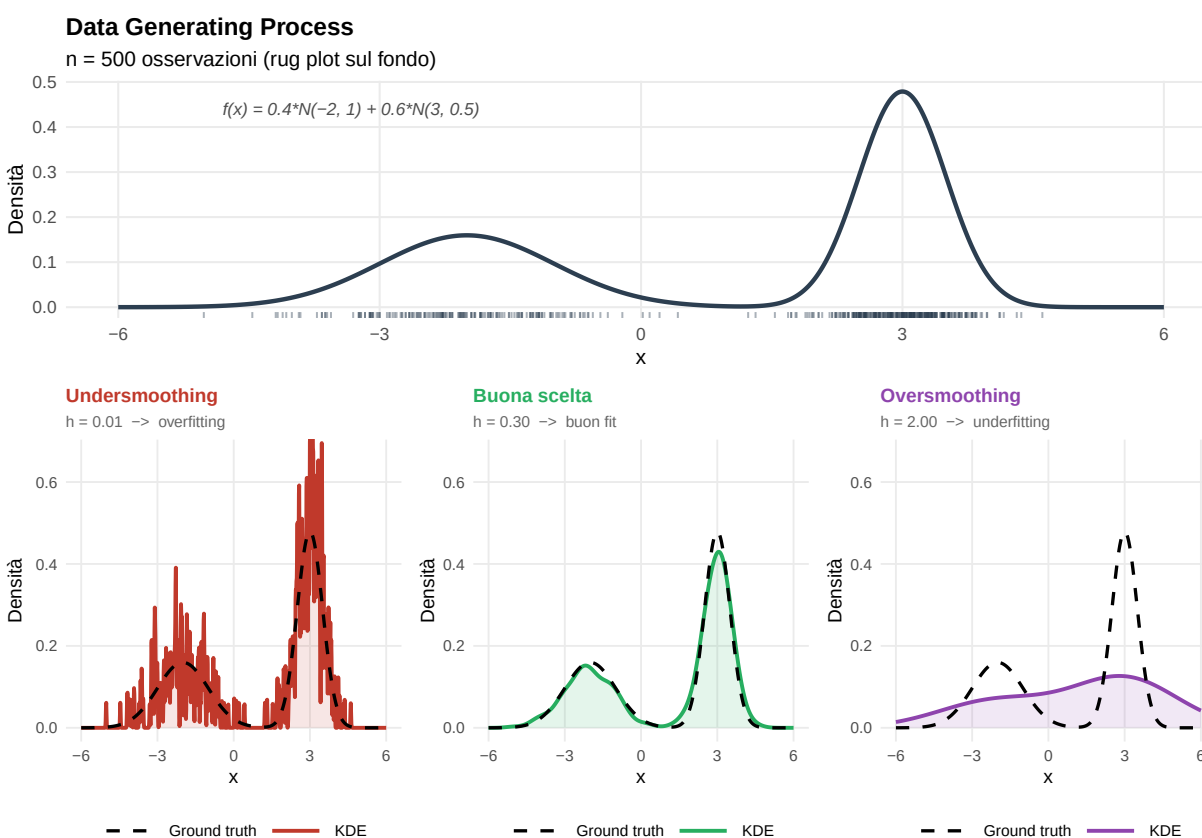
4.6 Layout combinato (4 pannelli — pronto per la slide)

```
layout_slide2 <- (p_dgp) /  
  (p_small | p_opt_panel | p_large) +  
  plot_annotation(  
    title = "KDE: Data Generating Process e variazione della bandwidth h",  
    subtitle = "Ground truth in tratteggio nero | n = 300",  
    theme = theme(  
      plot.title = element_text(face = "bold", size = 14),  
      plot.subtitle = element_text(size = 10.5, colour = "gray30")  
    )  
  ) +  
  plot_layout(heights = c(1.1, 1.4))
```

layout_slide2

KDE: Data Generating Process e variazione della bandwidth h

Ground truth in tratteggio nero | n = 300



5 Slide 3 — Confronto selettori di h e curva ISE

5.1 Obiettivo

La terza slide ha due pannelli affiancati:

- **Sinistra** — tabella con i valori di h prodotti da cinque metodi automatici più h^* (MISE empirico) come riferimento ottimale, con il relativo MISE (calcolabile qui perché conosciamo f).
- **Destra** — curva $\text{MISE}(h)$ con una linea verticale colorata per ciascun selettore e per h^* , così da vedere graficamente dove si posiziona ogni metodo rispetto al minimo della curva.

5.2 Calcolo dei selettori

Per ogni selettore calcoliamo il **MISE empirico** valutato esattamente nel valore di h suggerito — interpolando sulla griglia `h_seq` già simulata nella Slide 1.

```
h_methods <- c(
  "h* (MISE)"      = h_mise_opt,      # ottimo empirico - solo in contesto simulato
  "Silverman"      = bw.nrd(data),
  "Scott"          = bw.nrd(data),
  "Sheather-Jones" = bw.SJ(data),
  "UCV"            = bw.ucv(data),
  "BCV"            = bw.bcv(data)
)

# MISE di ciascun selettore: interpolazione lineare sulla griglia simulata
mise_methods <- sapply(h_methods, function(h) {
  approx(h_seq, mise_emp, xout = h)$y
})

# Data frame ordinato per MISE crescente
df_tab <- data.frame(
  Metodo = names(h_methods),
  h       = round(unname(h_methods), 4),
  MISE    = round(unname(mise_methods), 6),
  row.names = NULL
)
df_tab <- df_tab[order(df_tab$MISE), ]
```

5.3 Tabella formattata

```
if (has_kable_extra) {
  tab <- knitr::kable(
    df_tab,
    format      = "latex",
    booktabs    = TRUE,
    caption     = "Confronto tra selettori automatici di  $h$  e ottimo  $h^*$  (MISE empirico,  $B = 500$ )",
    escape      = FALSE,
    row.names   = FALSE,
    col.names   = c("Metodo", " $h$  stimato", "MISE")
  )
  tab <- kableExtra::kable_styling(
    tab,
    latex_options = c("striped", "hold_position"),
    full_width    = FALSE,
  )
}
```

Table 1: Confronto tra selettori automatici di h e ottimo h^* (MISE empirico, $B = 500$)

Metodo	h stimato	MISE
h^* (MISE)	0.1977	0.003214
UCV	0.1793	0.003245
BCV	0.2142	0.003292
Sheather-Jones	0.2156	0.003302
Silverman	0.6650	0.031295
Scott	0.7832	0.043037

```

font_size      = 11
)
tab <- kableExtra::row_spec(tab, 1, bold = TRUE, color = "white", background = "#27ae60")
kableExtra::footnote(
  tab,
  general = "La riga verde ( $h^*$ ) è l'ottimo empirico ottenuto minimizzando la curva MISE su  $B$  sim
  general_title = "Nota:",
  footnote_as_chunk = TRUE
)
} else {
  knitr::kable(
    df_tab,
    format      = "latex",
    booktabs    = TRUE,
    caption     = "Confronto tra selettori automatici di  $h$  e ottimo  $h^*$  (MISE empirico,  $B = 500$ )",
    escape      = FALSE,
    row.names   = FALSE,
    col.names   = c("Metodo", " $h$  stimato", "MISE")
  )
}

```

5.4 Curva MISE con linee verticali dei selettori

Il grafico mostra il **MISE empirico** (stimato con gli stessi B campioni della Slide 1) al variare di h , con una linea verticale per ciascun selettore automatico. Questo permette di vedere direttamente quanto ogni metodo si avvicina al minimo del MISE.

```

# Colori coerenti con la tabella (uno per metodo)
#  $h^*$  usa il nero per segnalare che è il riferimento teorico
col_met <- c(
  "h* (MISE)"      = "#1a252f",
  "Silverman"      = "#e74c3c",
  "Scott"          = "#e67e22",
  "Sheather-Jones" = "#27ae60",
  "UCV"            = "#3498db",
  "BCV"            = "#8e44ad"
)

# Riusa mise_emp e h_seq già calcolati nella Slide 1
# (stessa griglia h_seq, stessi B campioni con set.seed(123))
df_mise_plot <- data.frame(h = h_seq, MISE = mise_emp)

```

```

# Data frame per le linee verticali (include h*)
df_vlines <- data.frame(
  Metodo = names(h_methods),
  h      = unname(h_methods)
)
df_vlines$Metodo <- factor(df_vlines$Metodo, levels = names(col_met))

# Linee tratteggiate per i selettori automatici, linea solida per h*
df_vlines$linetype <- ifelse(df_vlines$Metodo == "h* (MISE)", "solid", "dashed")

# Tipi di tratteggio differenziati per migliorare la leggibilit 
# quando due selettori cadono su valori di h molto vicini.
lty_met <- c(
  "h* (MISE)"      = "solid",
  "Silverman"      = "longdash",
  "Scott"          = "dashed",
  "Sheather-Jones" = "dotdash",
  "UCV"            = "twodash",
  "BCV"            = "dotted"
)

# Punto di minimo MISE
h_mise_min <- h_seq[which.min(mise_emp)]
mise_min   <- min(mise_emp)

p_ise <- ggplot(df_mise_plot, aes(x = h, y = MISE)) +
  geom_line(colour = "#2c3e50", linewidth = 0.85) +
  # Linee verticali per i selettori automatici (tratteggiate)
  geom_vline(data = df_vlines[df_vlines$Metodo != "h* (MISE)", ],
    aes(xintercept = h, colour = Metodo, linetype = Metodo),
    linewidth = 0.85) +
  # Linea verticale per h* (solida, pi  spessa)
  geom_vline(data = df_vlines[df_vlines$Metodo == "h* (MISE)", ],
    aes(xintercept = h, colour = Metodo),
    linewidth = 0.85, linetype = "solid") +
  # Punto di minimo MISE
  geom_point(x = h_mise_min, y = mise_min,
    colour = "#1a252f", size = 3.5, shape = 16) +
  annotate("text",
    x = h_mise_min + 0.07, y = mise_min + diff(range(mise_emp)) * 0.06,
    label = paste0("h* = ", round(h_mise_min, 3)),
    size = 3.3, hjust = 0, colour = "#1a252f") +
  scale_x_continuous(limits = c(0, NA)) +
  scale_colour_manual(values = col_met, name = "Selettore / Riferimento") +
  scale_linetype_manual(values = lty_met, guide = "none") +
  labs(
    title      = "MISE empirico in funzione di h - curva a U",
    subtitle   = paste0("n = ", n, " | B = ", B,
      " simulazioni | f(x) = 0.4 N(-2, 1) + 0.6 N(3, 0.5)"),
    x          = "h (bandwidth)",
    y          = "MISE"
  ) +
  theme_minimal(base_size = 13) +

```

```

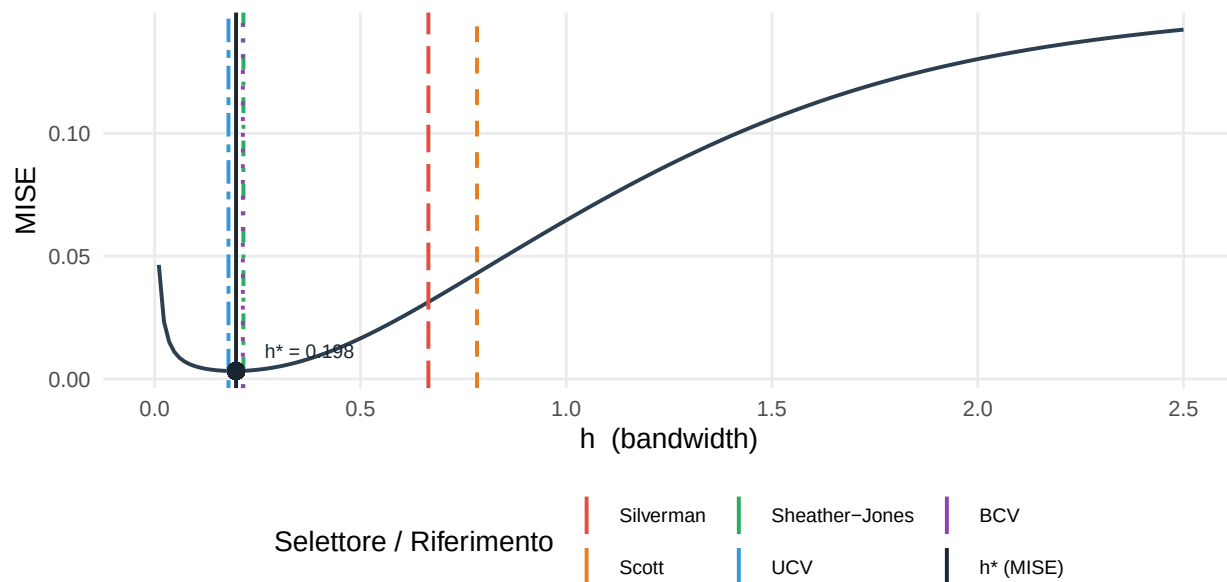
theme(
  plot.title      = element_text(face = "bold"),
  legend.position = "bottom",
  legend.text     = element_text(size = 9),
  panel.grid.minor = element_blank()
) +
guides(colour = guide_legend(nrow = 2))

```

p_ise

MISE empirico in funzione di h – curva a U

$n = 500$ | $B = 100$ simulazioni | $f(x) = 0.4 N(-2, 1) + 0.6 N(3, 0.5)$



Lettura del grafico. La curva a U ha il minimo (punto nero) in corrispondenza dell' h ottimale per il MISE stimato su B simulazioni — questa è la quantità teoricamente rilevante, a differenza dell'ISE su un singolo campione che dipende dalla realizzazione specifica. Le linee verticali mostrano dove si posiziona ogni selettore: Silverman e Scott (rosso/arancio) tendono a stare *a destra* del minimo, confermando la loro tendenza all'oversmoothing su distribuzioni non gaussiane; Sheather-Jones e i metodi cross-validation si avvicinano maggiormente al minimo.

Table 2: Riepilogo: selettori di h a confronto con l'ottimo h^* (MISE empirico)

Metodo	h	MISE	Distanza da h^*	MISE / MISE min
h^* (MISE)	0.1977	0.003214	0.0000	1.000
UCV	0.1793	0.003245	0.0184	1.010
BCV	0.2142	0.003292	0.0165	1.024
Sheather-Jones	0.2156	0.003302	0.0179	1.027
Silverman	0.6650	0.031295	0.4673	9.736
Scott	0.7832	0.043037	0.5855	13.390

6 Riepilogo

```
# Tabella riepilogativa finale: selettori con distanza da  $h^*$  MISE e rapporto MISE
df_final <- df_tab
df_final$"Distanza da  $h^*$ " <- round(abs(df_final$h - h_mise_min), 4)
df_final$"MISE / MISE min" <- round(df_final$MISE / mise_min, 3)

if (has_kable_extra) {
  tab_final <- knitr::kable(
    df_final,
    format = "latex",
    booktabs = TRUE,
    caption = "Riepilogo: selettori di  $h$  a confronto con l'ottimo  $h^*$  (MISE empirico)",
    row.names = FALSE
  )
  tab_final <- kableExtra::kable_styling(
    tab_final,
    latex_options = c("striped", "hold_position"),
    full_width = FALSE,
    font_size = 10
  )
  kableExtra::row_spec(tab_final, 1, bold = TRUE)
} else {
  knitr::kable(
    df_final,
    format = "latex",
    booktabs = TRUE,
    caption = "Riepilogo: selettori di  $h$  a confronto con l'ottimo  $h^*$  (MISE empirico)",
    row.names = FALSE
  )
}
```

Osservazione principale. Su questa distribuzione bimodale:

- h^* (MISE empirico) è il **riferimento ottimale**: ha distanza 0 dall'ottimo e rapporto $\text{MISE}/\text{MISE}_{\min} = 1.000$ per definizione. È calcolabile solo perché siamo in contesto simulato e conosciamo f .
- Silverman e Scott sovrastimano h perché $\hat{\sigma}$ è gonfiata dalla distanza tra i due picchi (~ 5 unità), producendo un MISE significativamente superiore al minimo.
- Sheather-Jones stima h più vicino all'ottimale, poiché il suo procedimento plug-in tiene conto della curvatura locale della distribuzione.
- UCV e BCV minimizzano direttamente un criterio legato al MISE stimato sul campione tramite leave-one-out, ma possono essere instabili (UCV in particolare può avere minimi multipli per campioni

piccoli).

Per distribuzioni semplici (unimodali, approssimativamente gaussiane) Silverman è un ottimo punto di partenza. Per distribuzioni complesse, preferire Sheather–Jones o cross-validation.